A Graph-Based Iterative Strategy for Solving the All-Line Transmission Switching Problem

SEIO 2025 (June 11th)

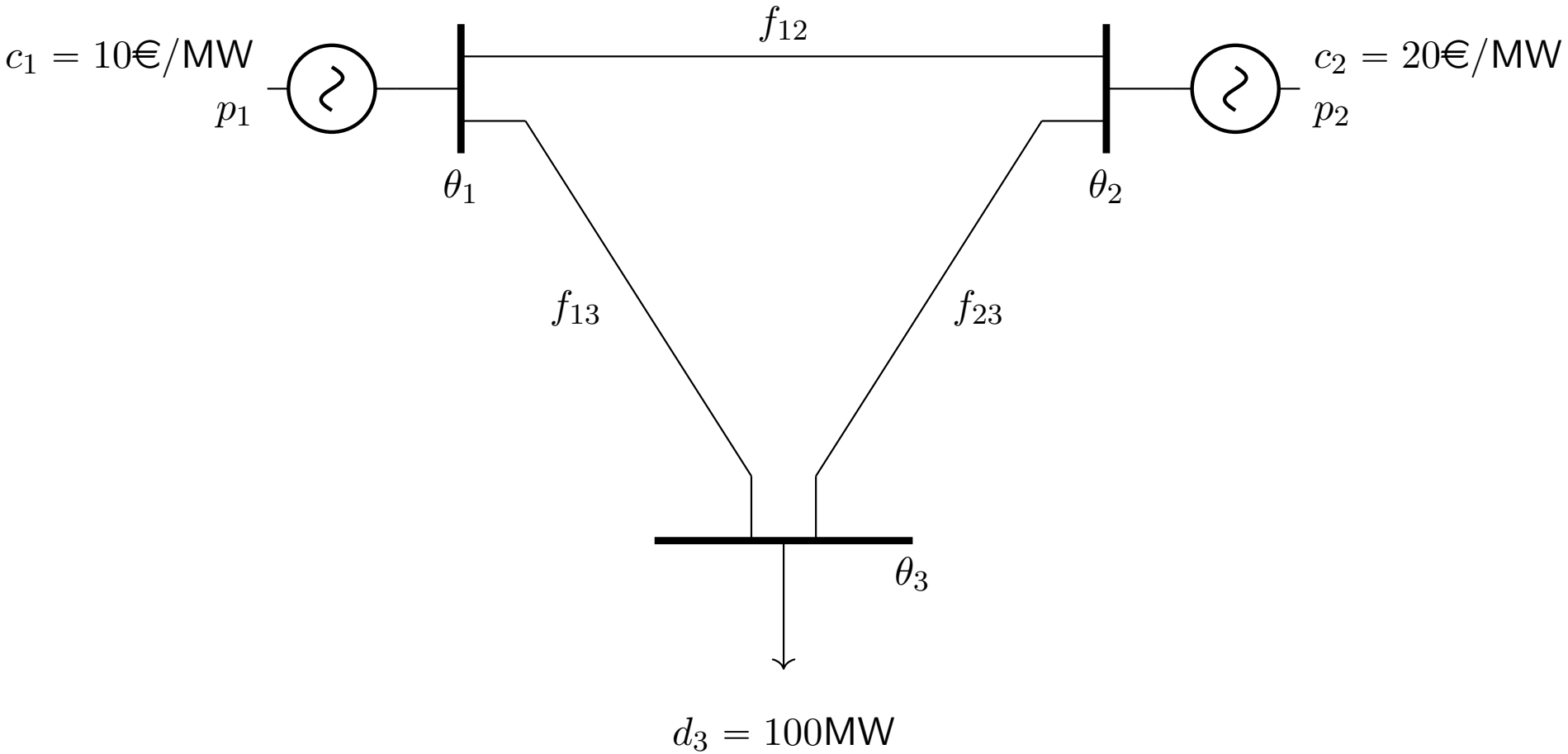
Salvador Pineda

(joint work with M. Aguilar-Moreno, J. M. Morales)

OASYS group, University of Málaga (Spain)

Motivating example

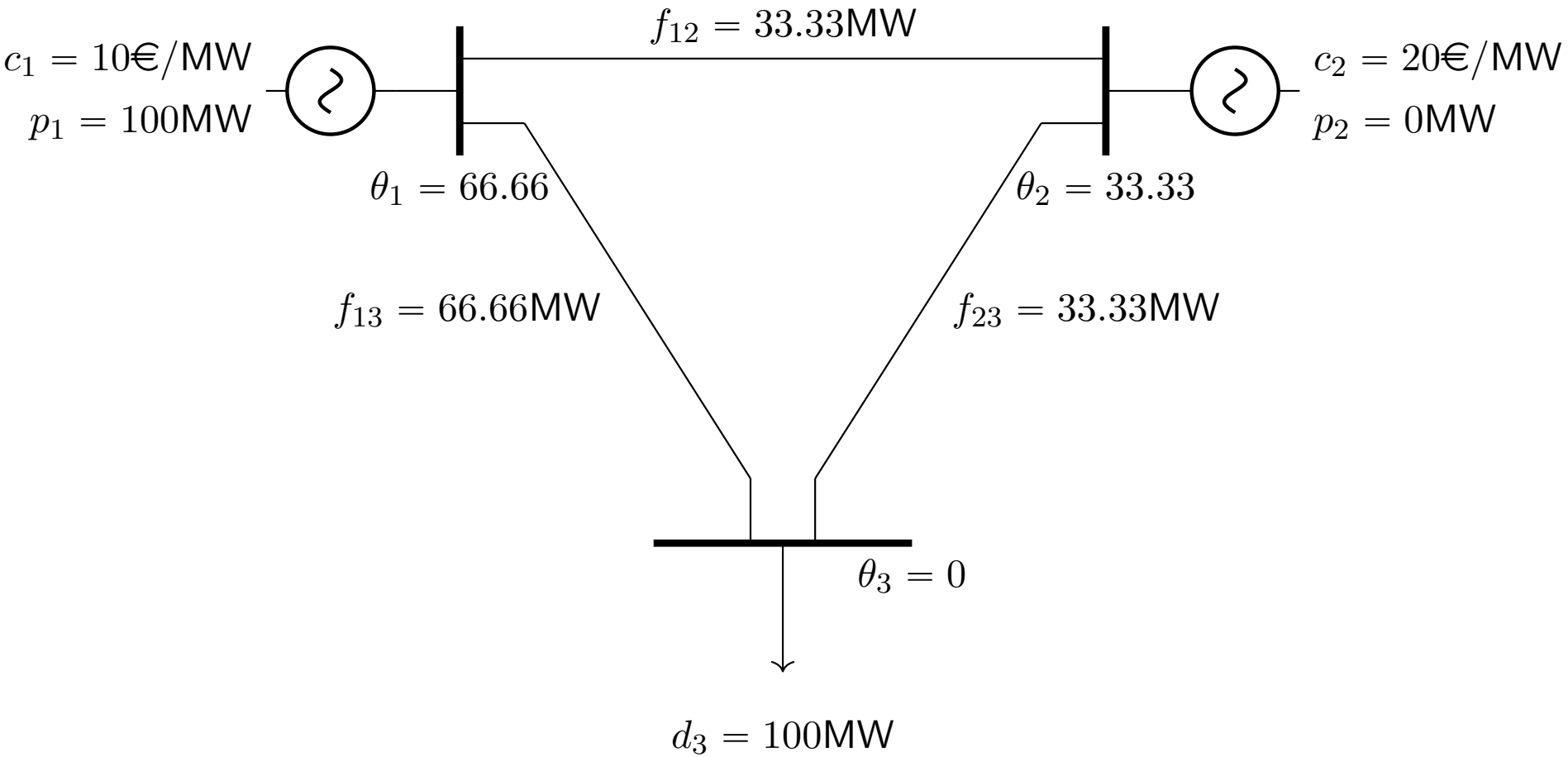
Optimal power flow (OPF): Determine the power generation and power flows to satisfy the demand at the minimum cost



The cheap unit satisfies the demand

Motivating example

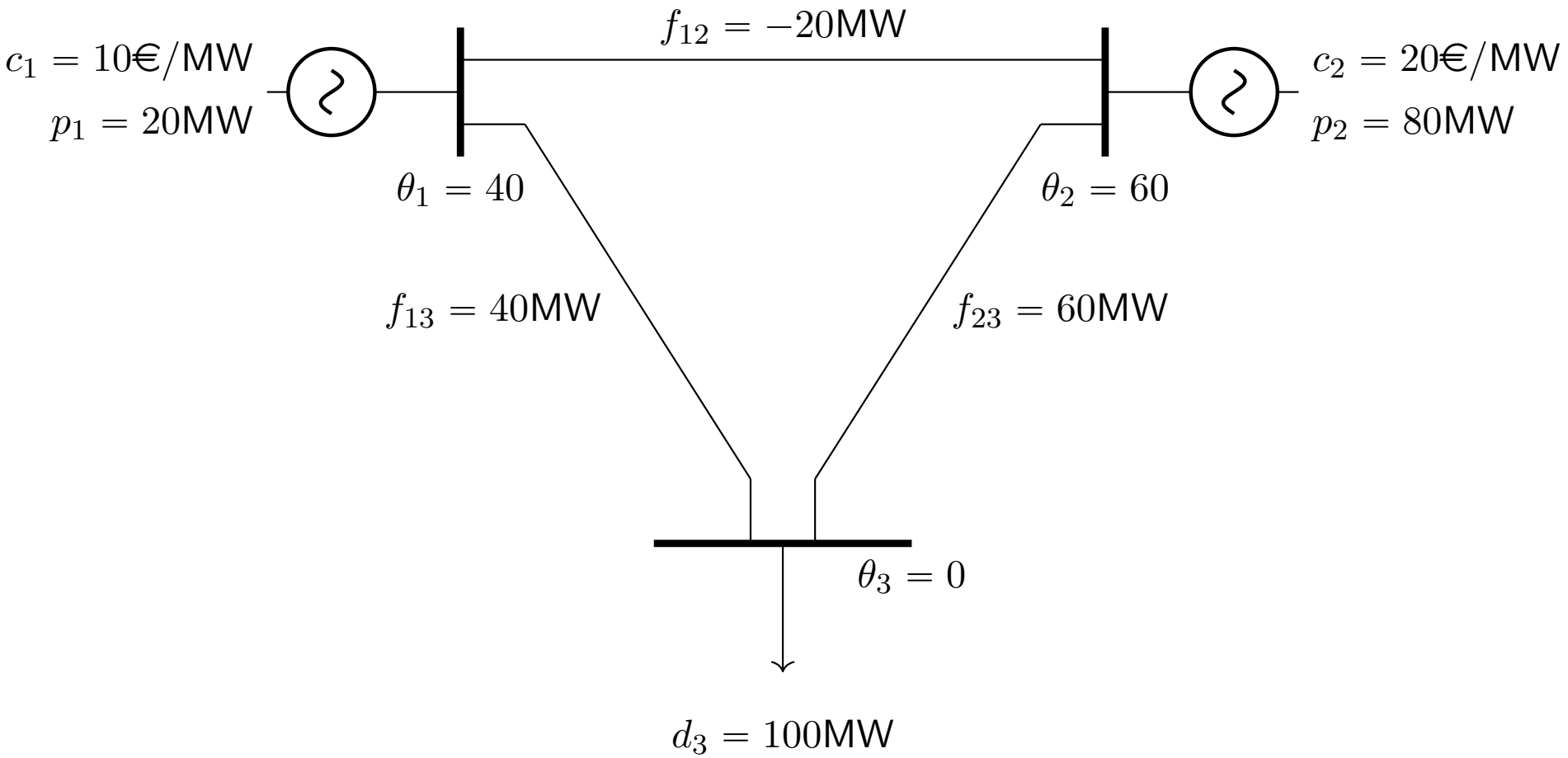
Optimal solution: generate all with cheapest unit (cost = 1000€)



Electrons are not potatoes!!!

Motivating example

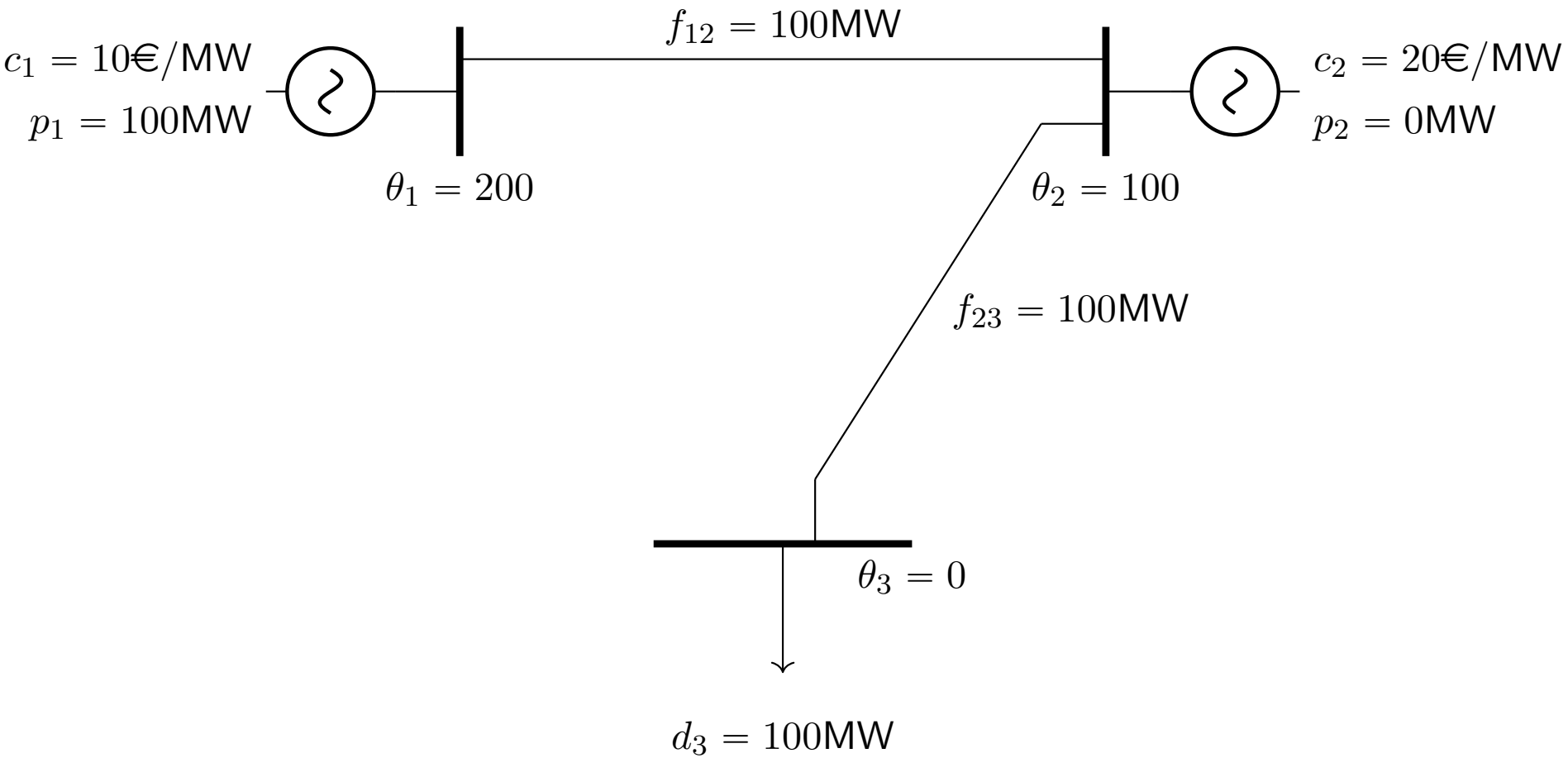
If $f_{13} \leqslant 40$, the expensive unit also generates (cost=1800€)



Network limits increase the cost!!!

Motivating example

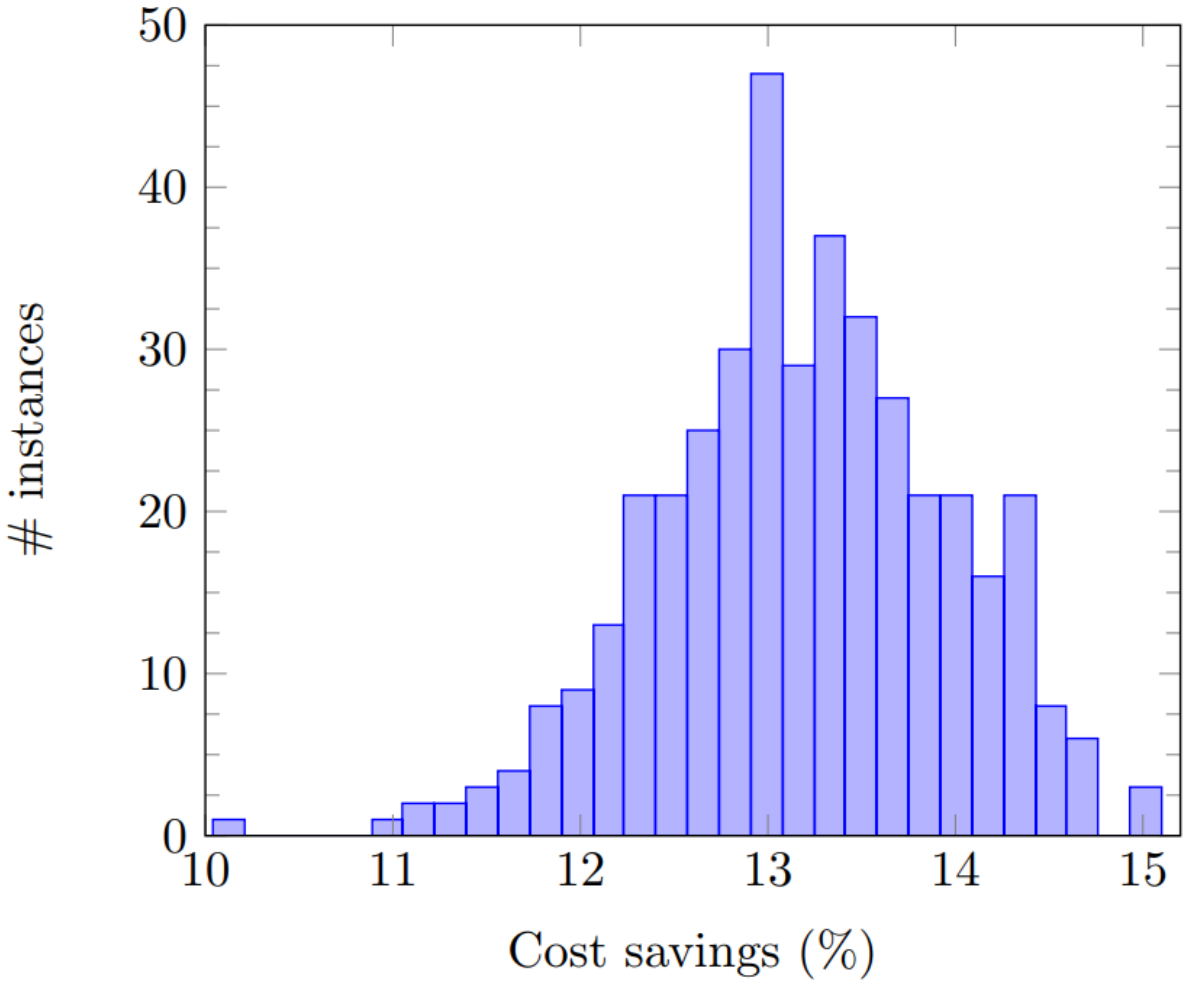
If line 13 is disconnected, cost= 1000€



Disconnecting lines can reduce cost!!!

Motivating example

In the 118-bus system, the average cost saving is 13.2%



Formulation

The optimal power flow (OPF) is formulated as a linear optimization problem

$$\min_{p_i, f_{ij}, \theta_i} \quad \sum_i c_i \, p_i \qquad \text{(minimize generation cost)} \tag{1a}$$

s.t.

$$\text{(power balance)} \quad \sum_{(i,j) \in \mathcal{L}_i^-} f_{ij} - \sum_{(i,j) \in \mathcal{L}_i^+} f_{ij} = p_i - d_i, \quad \forall i \tag{1b}$$

$$\text{(power flow)} \quad f_{ij} = b_{ij}(\theta_i - \theta_j), \quad \forall (i,j) \in \mathcal{L} \tag{1c}$$

$$\text{(generation limits)} \quad \underline{p}_i \leqslant p_i \leqslant \overline{p}_i, \quad \forall i \tag{1d}$$

$$\text{(power flow limits)} \quad -\underline{f}_{ij} \leqslant f_{ij} \leqslant \overline{f}_{ij}, \quad \forall (i,j) \in \mathcal{L} \tag{1e}$$

Formulation

The optimal transmission switching (OTS) requires binary variables x_{ij} and is formulated as a mixed-integer non-linear problem ...

$$\min_{p_i, f_{ij}, \theta_i, x_{ij}} \quad \sum_i c_i \, p_i \tag{2a}$$

$$\text{s.t.} \quad \sum_{(i,j) \in \mathcal{L}_i^-} f_{ij} - \sum_{(i,j) \in \mathcal{L}_i^+} f_{ij} = p_i - d_i, \quad \forall i \tag{2b}$$

$$f_{ij} = x_{ij} b_{ij} (\theta_i - \theta_j), \quad \forall (i,j) \in \mathcal{L} \tag{2c}$$

$$\underline{p}_i \leqslant p_i \leqslant \overline{p}_i, \quad \forall i \tag{2d}$$

$$-x_{ij} \underline{f}_{ij} \leqslant f_{ij} \leqslant x_{ij} \overline{f}_{ij}, \quad \forall (i,j) \in \mathcal{L} \tag{2e}$$

$$x_{ij} \in \{0, 1\}, \quad \forall (i,j) \in \mathcal{L} \tag{2f}$$

... that can be directly solved using optimization solvers such as Gurobi.

Formulation

The OTS is reformulated as a mixed-integer linear problem

$$\min_{p_i, f_{ij}, \theta_i, x_{ij}} \quad \sum_i c_i \, p_i \tag{3a}$$

$$\text{s.t.} \quad \sum_{(i,j) \in \mathcal{L}_i^-} f_{ij} - \sum_{(i,j) \in \mathcal{L}_i^+} f_{ij} = p_i - d_i, \quad \forall i \tag{3b}$$

$$b_{ij}(\theta_i - \theta_j) - \overline{M}_{ij}(1 - x_{ij}) \leqslant f_{ij}, \quad \forall (i,j) \in \mathcal{L} \tag{3c}$$

$$f_{ij} \leqslant b_{ij}(\theta_i - \theta_j) - \underline{M}_{ij}(1 - x_{ij}), \quad \forall (i,j) \in \mathcal{L} \tag{3d}$$

$$\underline{p}_i \leqslant p_i \leqslant \overline{p}_i, \quad \forall i \tag{3e}$$

$$-x_{ij}\underline{f}_{ij} \leqslant f_{ij} \leqslant x_{ij}\overline{f}_{ij}, \quad \forall (i,j) \in \mathcal{L} \tag{3f}$$

$$x_{ij} \in \{0,1\}, \quad \forall (i,j) \in \mathcal{L} \tag{3g}$$

$\underline{M}_{ij}$ and $\overline{M}_{ij}$ must be valid bounds for $b_{ij}(\theta_i - \theta_j)$ if line is open

$\underline{M}_{ij}$ and $\overline{M}_{ij}$ must be small enough to avoid computational issues

Spanning Tree and Connectivity

- **1. With a spanning tree of connected lines Fattahi et al. (2019)**
  - Guarantees network connectivity by design.
  - Fewer binary variables (only some lines can be disconnected).
  - Big-M constants can be computed efficiently via **shortest-path**.
- **2. All lines switchable, no connectivity (islanding allowed)**
  - The model may result in disconnected (islanded) solutions.
  - Requires binary variables for **all** lines.
  - Big-M computed using the **longest-path** problem (NP-hard).
- **3. All lines switchable, with connectivity constraints**
  - Maintains connectivity explicitly via extra constraints.
  - Still requires binary variables for all lines.
  - Big-M values computed via **longest-path** problem (NP-hard).
  - Commonly used in practice for stability reasons.

Contributions

Context: We focus on the most challenging OTS setting, where **all lines are switchable** and **network connectivity must be enforced**.

Main contributions:

- A **graph-based algorithm** to compute tighter voltage angle bounds (big-M values).
- A **heuristic method** to efficiently generate high-quality feasible solutions.
- An **iterative strategy** that integrates both techniques to reduce mixed-integer solver time.
- Extensive **validation on the IEEE 118-bus system** showing performance improvements.

Greedy Upper Bound for Longest Path

Greedy method from Moulin et al. (2010):

- Computes upper bound of longest path based on given bus ordering.
- Iteratively connects each bus (not yet connected to previous ones) to an already connected bus using the edge with the largest weight.
- Continues until all buses are connected to the initial subgraph.
- Limitation: may yield values smaller than the true longest path.
- Longest path $n_4 - n_5$ is 162, bound by greedy 142 (not valid).

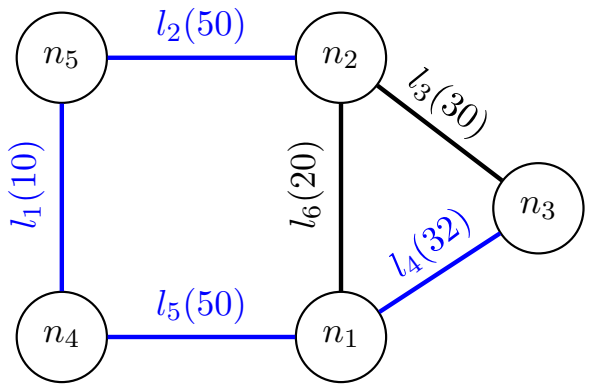
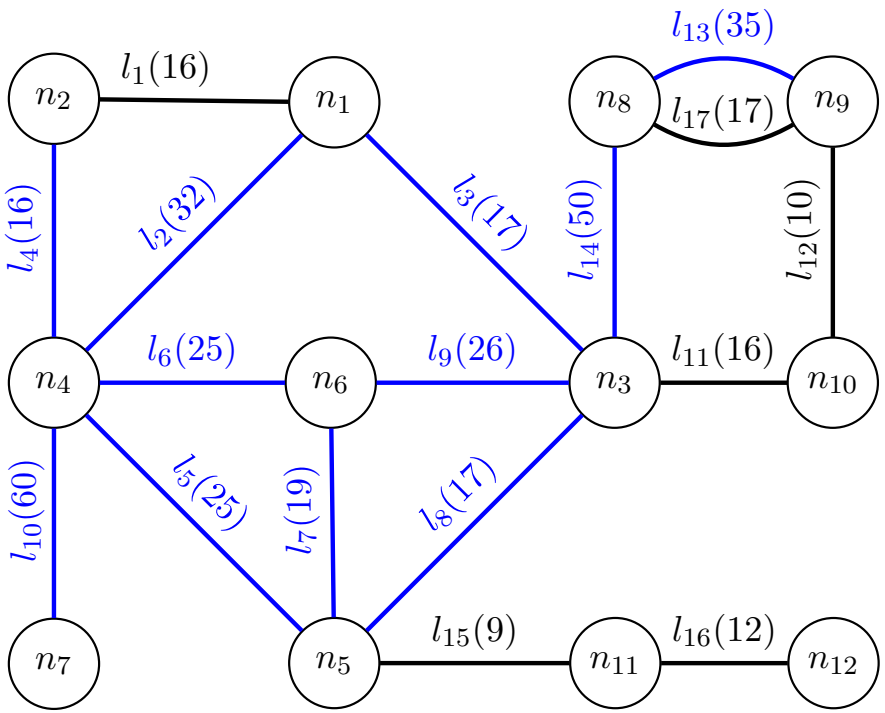


Figure: Counterexample for methodology proposed in Moulin et al. (2010).

Naive Upper Bounds on Longest Path

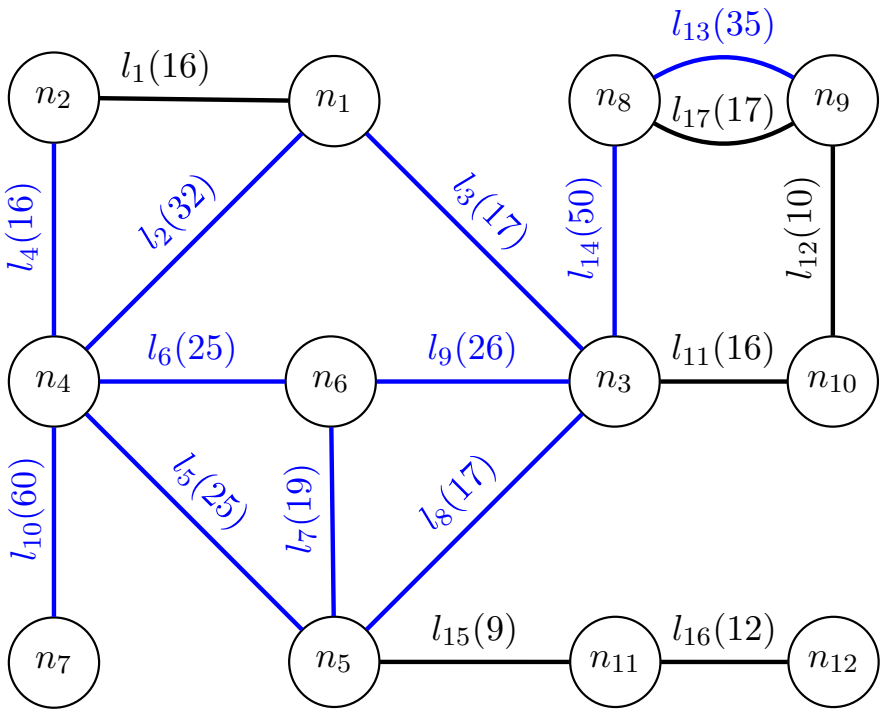
- Sum of all edge weights (a bit silly).
- Sum of the $|\mathcal{N}| - 1$ largest edge weights.
- Typically loose and not graph-aware.



$$M_{l_1} = b_{l_1} \cdot (60 + 50 + 35 + \cdots + 17 + 17 + 16) = b_{l_1} \cdot 322$$

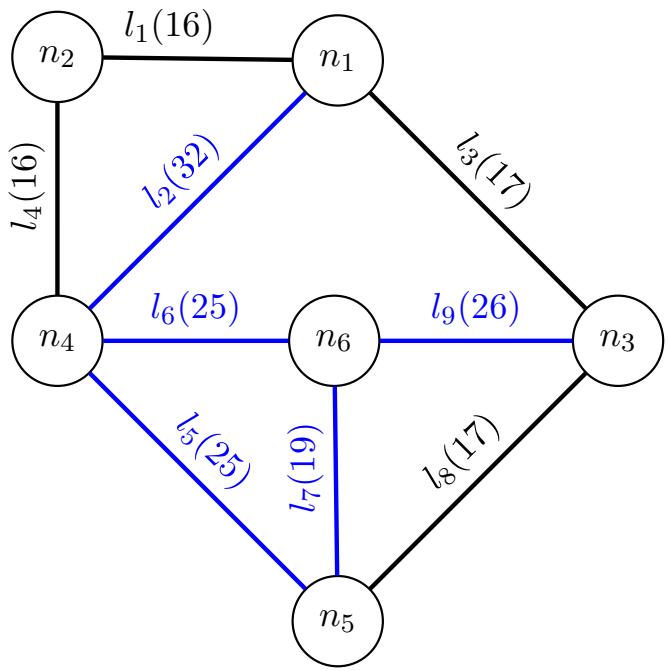
Tighter big-Ms via Graph Simplification

- The leaf edge l_{10} cannot be part of longest path.
- Node n_3 is a cut node and l_{13} and l_{14} cannot be part of longest path.



Tighter big-Ms via Graph Simplification

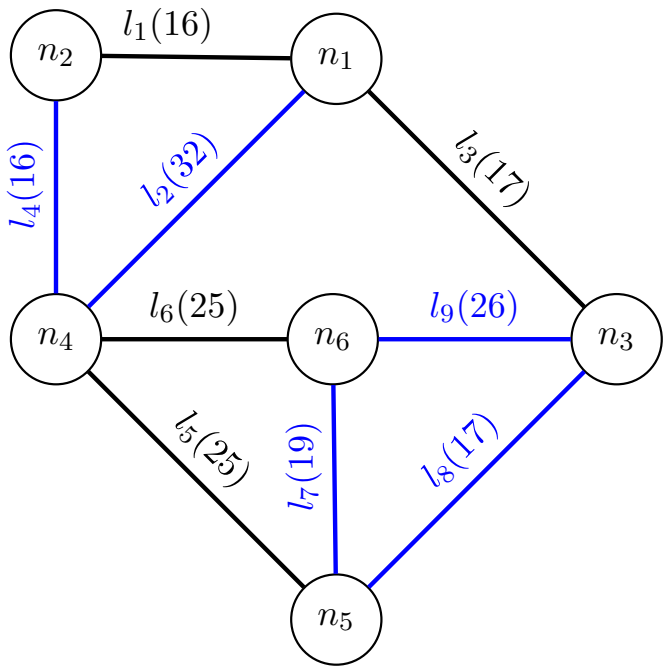
- Leaf edges are removed, graph is divided into subgraphs using cut nodes, and naive approach is applied.
- Some nodes are connected to more than two lines (not a path).

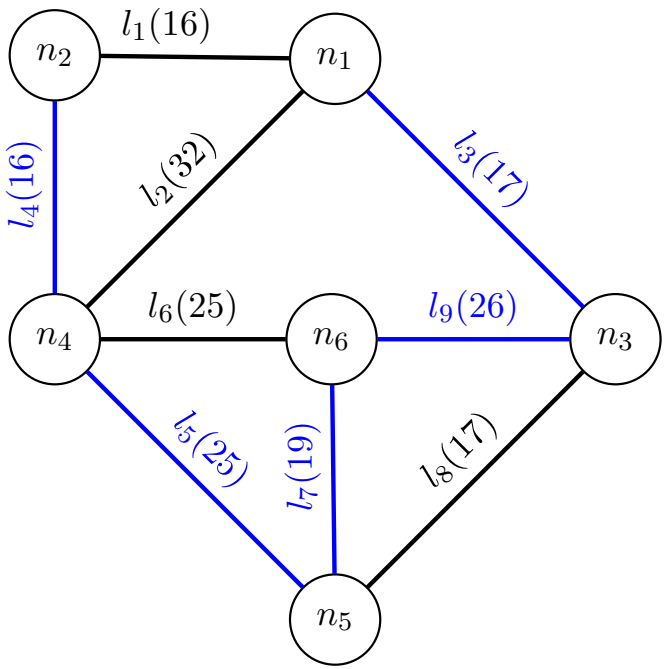


$$M_{l_1} = b_{l_1} \cdot (32 + 26 + 25 + 25 + 19) = b_{l_1} \cdot 127 \quad (60\% \text{ reduction})$$

Tighter big-Ms via Relaxed Longest Path

- After applying graph simplifications, we compute an upper bound on big-Ms by solving a **relaxed version** of the longest path problem that ignores subtour elimination constraints.
- Computational burden is significally reduced.





- Relaxed Longest Path: $M_{l_1} = b_{l_1} \cdot 110$ $\quad (66\%$ reduction$)$
- Original Longest Path: $M_{l_1} = b_{l_1} \cdot 103$ $\quad (68\%$ reduction$)$

Tighter big-Ms via Graph Simplification & Relaxed LP

The proposed approach to tighten big-Ms run as follows:

1. **Remove leaf edges**.
2. **Decompose at cut nodes and create subgraphs**.
3. **Solve relaxed longest-path problem**.

We compare the bigMs of the benchmark (BN), the longest path (LP) and the proposed simplify+relax (SR) using:

$$\lambda_l^{BN} = \frac{M_l^{BN}}{M_l^{LP}} \qquad \lambda_l^{SR} = \frac{M_l^{SR}}{M_l^{LP}} \qquad \text{where } \lambda_l^{BN} \geqslant \lambda_l^{SR} \geqslant 1$$

Summary statistics for λ:

- λ_l^{BN}: min $= 1.88$, avg $= 3.23$, max $= 24.26$
- λ_l^{SR}: min $= 1.00$, avg $= 1.12$, max $= 1.45$

Computational times:

- SR average: 0.007 s
- LP average: 4534 s (up to 24 hours, memory limit issues)

Heuristic-Driven Iterative OTS

Idea: Use partial progress from early solver runs to intelligently simplify and strengthen the OTS formulation to improve best incumbent.

Workflow:

1. **Start solving the full OTS** for a limited time T_i^O.
2. **Collect best solutions found so far**: top-$|\mathcal{K}_i|$ incumbents.
3. **Analyze patterns:**
   - Fix binaries for lines always connected.
   - Use solution graphs to compute **tighter big-Ms** via shortest paths.
4. **Build a simpler OTS**: fewer binaries, stronger relaxations.
5. **Solve the simplified model** for time T_i^H to improve incumbent.
6. **Use new best solution** to warm-start the next full OTS round.

Benefits:

- Reduces complexity in early iterations.
- Extracts more value from intermediate solutions.
- Iteratively improves both solution quality and solver performance.

Heuristic-Driven Iterative OTS

	Benchmark (BN)		Proposed (SR)		LongestPath (LP)	
Model	ObjVal	Gap	ObjVal	Gap	ObjVal	Gap
Original	1496.4	5.88%	1468.6	4.07%	1505.9	6.47%
Reduced	**1468.3**	0.79%	**1456.7**	-	**1481.4**	-
Original	1454.8	0.43%	1456.7	0.68%	1448.8	0.02%
Reduced	**1453.6**	-	1456.7	-	**1448.6**	-
Original	1450.5	0.14%	1448.7	0.01%	1448.6	-
Reduced	**1450.4**	-	**1448.6**	-		
Original	1450.3	0.12%	1448.6	-		
Reduced	1450.3	-				
Original	1448.8	0.02%				
Reduced	1448.8	-				
Original	1448.6	-				
Time SS	3600s		1336s		331s	
Time IT	1595s		243s		163s	

Comparison of Methodologies

Objective: Compare six methodologies for solving the OTS problem where all lines are switchable and connectivity is enforced.

Big-M Computation Methods:

- **BN (Benchmark):** Adds the largest $|N| - 1$ weights.
- **LP (Longest Path):** Solves NP-hard problems for each line.
- **SR (Simplify + Relax):** Graph simplifications and relaxed LP.

Solution Strategies:

- **SS (Single Step):** Solve optimization model directly.
- **IT (Iterative):** Use proposed iterative procedure.

Evaluated Combinations:

BN-SS, BN-IT, LP-SS, LP-IT, SR-SS, SR-IT

Numerical Simulations

Test Case: IEEE 118-bus system with 186 lines.

- 100 demand scenarios: uniform sampling in $[0.9\widehat{d}_n, 1.1\widehat{d}_n]$.
- Iterative OTS: original model time limit increases per round (30s to 600s), heuristic fixed at 30s.
- Top 10 feasible solutions used at each iteration.
- Solved with GUROBI 10.0.3 (1 thread, 2.6 GHz CPU, 8 GB RAM).
- Optimality gap 0.01%, maximum time 1 hour.
- Full OTS achieves average cost savings of 10.35% over systems with given spanning trees.

Numerical Results

Method	Avg. time (s)	Max Gap	Avg. Gap	# Unsolved
BN-SS	1893.57	2.91%	0.18%	47
SR-SS	1660.14	7.27%	0.18%	40
LP-SS	2133.58	2.72%	0.19%	55
BN-IT	1234.16	0.43%	0.03%	28
SR-IT	**1054.95**	**0.39%**	**0.02%**	**20**
LP-IT	1128.42	0.43%	**0.02%**	26

- The simplify+relax approach to tighten big-M values improves average solution quality.
- Computing the tightest big-Ms via the longest path is computationally expensive and does not necessarily yield the best results.
- Regardless of the big-M values used, the proposed iterative approach consistently improves solution quality.
- Combining the simplify+relax method with the iterative approach produces the best overall results.

Conclusions

- Optimal transmission switching (OTS) reduces costs by adapting network topology.
- Large big-M values hurt solver performance; tight big-Ms are hard to compute for fully switchable lines.
- Proposed method:
  - Simplify network and solve relaxed longest-path for tighter big-Ms.
  - Iterative heuristic uses best solutions to improve best incumbent.
- Results on IEEE 118-bus:
  - 57% fewer unsolved cases.
  - 44% less average solving time.

Thanks for the attention!! Questions??

PREPRINT AVAILABLE:

A Graph-Based Iterative Strategy for Solving the All-Line Transmission Switching Problem

M. Aguilar-Moreno, S. Pineda, *Senior Member, IEEE,* J. M. Morales, *Senior Member, IEEE*

OASYS
Optimization and Analytics for Sustainable energY Systems

More info: oasys.uma.es

Email: spineda@uma.es

References

Salar Fattahi, Javad Lavaei, and Alper Atamtürk. A bound strengthening method for optimal transmission switching in power systems. *IEEE Transactions on Power Systems*, 34(1):280–291, 2019. doi: 10.1109/TPWRS.2018.2867999.

Luciano S Moulin, Michael Poss, and Claudia Sagastizábal. Transmission expansion planning with re-design. *Energy systems*, 1:113–139, 2010.